

Assignment 1

Coffee Data Analysis



Report By:

Vivek Chinmay Dhanwada

47895403

Report For:

COMP6210

Big Data

Session 1 2025

Flowcharts and Pseudocodes

1. Task1_2.py

Begin MapReduce Job: Total Coffee Sales

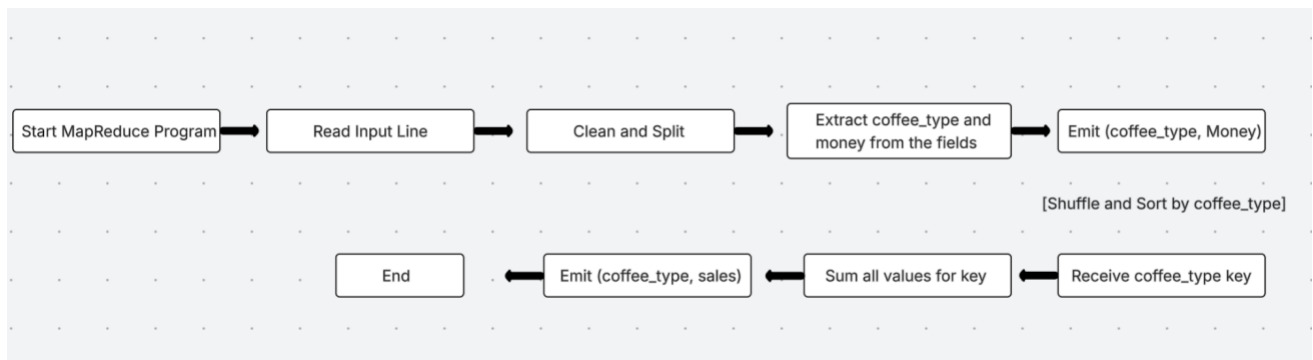
Mapper Function:

```
For each line in input:  
    Clean the line (remove braces and quotes)  
    Extract coffee_type and money from the line  
    If coffee_type and money are found:  
        Emit (coffee_type, money)
```

Reducer Function:

```
For each coffee_type key:  
    Sum all money values  
    Emit (coffee_type, total_sales_rounded_to_2dp)
```

End MapReduce Job



Task 2_1.py

Start MapReduce Program

Mapper Step:

```
For each line in input:  
    Clean and split the line  
    Extract coffee_type and money  
    If coffee_type is found:  
        Emit (coffee_type, money)
```

Reducer Step 1:

```
For each coffee_type:  
    Sum all money values
```

```

        Emit (None, (total_sales, coffee_type))    # Emit total first
to enable sorting

```

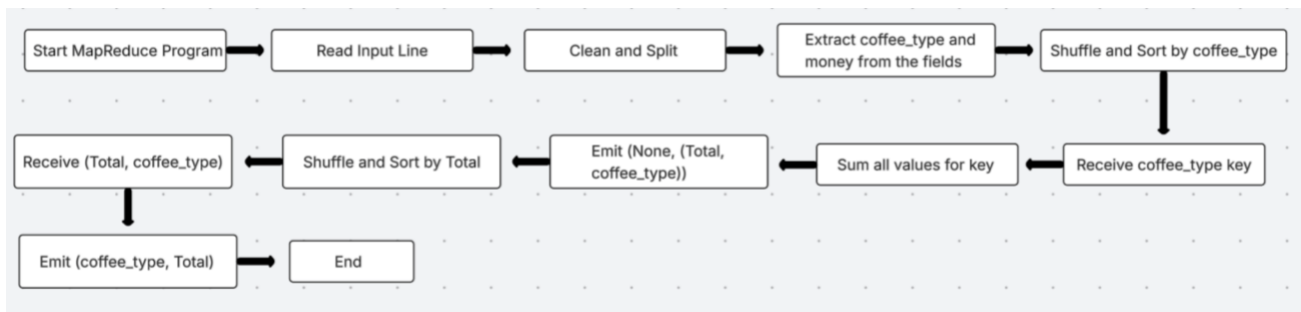
Reducer Step 2:

```

    Receive all (total_sales, coffee_type) pairs
    Sort all pairs by total_sales (ascending)
    For each sorted pair:
        Emit (coffee_type, total_sales)

```

End



Task2_1_monthly.py

Start MapReduce Program

Mapper:

```

    For each input line:
        Clean the line and split into fields
        Extract coffee_type, money, and month
        If all three exist:
            Emit ((coffee_type, month), money)

```

[Shuffle and Sort by (coffee_type, month)]

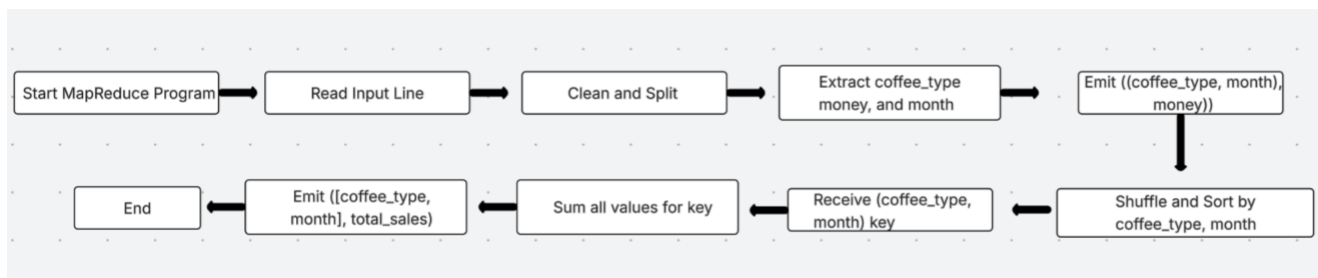
Reducer:

```

    For each (coffee_type, month) key:
        Sum all values
        Emit ([coffee_type, month], total_sales)

```

End Program



Task2_2.py

Start MapReduce Program

Mapper:

```

For each line in the input:
    Clean and split the line
    Extract user_id and money
    If user_id is valid:
        Emit (user_id, money)
  
```

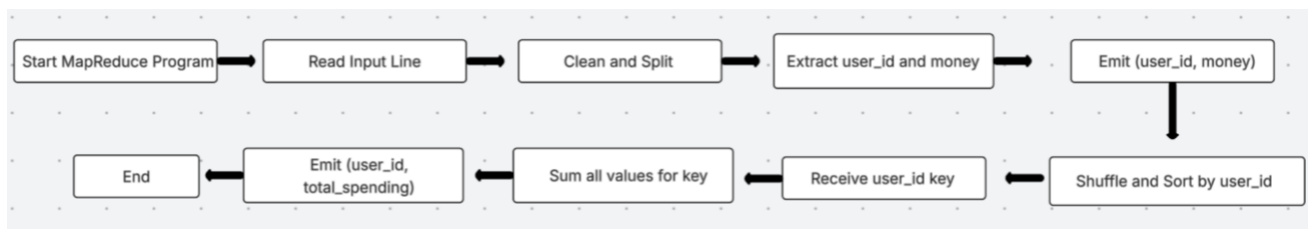
[Shuffle and Sort by user_id]

Reducer:

```

For each user_id key:
    Sum all money values
    Emit (user_id, total_spending)
  
```

End



Task2_2_monthly.py

Start MapReduce Program

Mapper:

```

For each input line:
    Clean and split the line
    Extract user_id, money, and month
    If user_id and month are present:
  
```

```

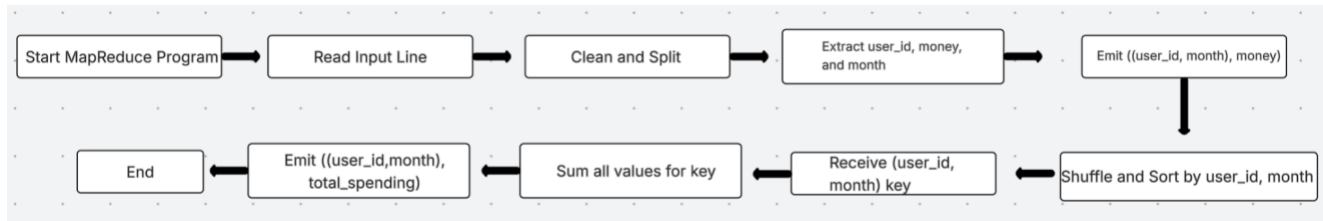
    Emit ((user_id, month), money)

[Shuffle and Sort by (user_id, month)]

Reducer:
    For each (user_id, month) key:
        Sum all money values
        Emit ((user_id, month), total_spending)

End

```



Code Explanation

Overview of My Work

This project focused on analysing a coffee sales dataset using MapReduce to extract meaningful patterns and insights. The tasks were split into two main areas: analysing coffee types and analysing customer behaviour. I used the mrjob Python library to simulate the MapReduce process, and followed the same code structure we used in class so it remains simple and easy to understand. Below is an explanation of what I did in each task.

Task 1.1 – Data Cleaning and Triplet Extraction

The first step was to clean the original data and prepare it for further analysis. I used a Python script with Pandas and PyMongo to:

- Drop any rows that had missing data.
- Keep only transactions paid with a card.

Extract the following details for each valid transaction: coffee_type, user_id (which is the card number), money spent, and the day, month, year, and time of the transaction. These values were stored as a dictionary (or triplet) and saved into a text file and a new MongoDB collection. This cleaned data was then used for all the MapReduce tasks.

Task 1.2 – Total Sales per Coffee Type

In this task, I calculated how much revenue was made from each coffee type. I wrote a MapReduce program that reads each transaction, finds the coffee type and how much money was spent, and then adds them all up. This helped me understand which coffees were most popular by total sales.

Task 2.1 – Total and Monthly Sales of Bottom 3 Coffee Types

After calculating the total sales of each coffee in Task 1.2, I modified the MapReduce program to sort all coffee types from lowest to highest total sales. This helped identify which ones were selling the least overall. Then, I picked the three coffee types with the lowest total sales and wrote another MapReduce program to calculate how much of each was sold in each month. I used this monthly output to create a line chart, so I could visually track the sales trend of the bottom 3 coffees across the year. This helped me understand if their low performance was consistent or seasonal.

Task 2.2 – Total Spend per Customer and Monthly Pattern of Top 5 Customers

Here, I looked at customer behaviour. I created a MapReduce program to find out how much money each customer spent in total. Each customer was identified by their card number. Then I used a small Python script to sort all the customers and select the top 5 biggest spenders. This helped identify who were the most valuable customers overall.

Once I had the top 5 customers, I created another MapReduce script to calculate their spending month by month. I then used the output to make a line chart comparing the monthly spend pattern of the top 5 customers. This showed if they were regular buyers or had high spending in certain months only.

Final Thoughts

Throughout this assignment, I focused on keeping my code beginner-friendly and following the class style. I reused similar logic across tasks and added only what was necessary for each new question. I also made sure my charts were clear and matched what the assignment asked for. Flowcharts and pseudocode were included to make the logic of each program easier to understand.